Aprueba · Sitio web del alumno (full-stack)

Implementacion del **sitio web del estudiante** de Aprueba a partir del wireframe `Aprueba_Web_Wireframe.html`, el contrato de `Aprueba_API_Backend.docx` (seccion 1, "API del alumno") y el modelo de datos de `Aprueba_Modelo_Datos_Firestore.docx`.

- **Frontend:** React 18 + Vite + React Router (JavaScript). Estilos portados 1:1 del wireframe (tokens de marca, tema claro/oscuro, ES/EN). Login, registro, recuperar contrasena, social (demo), onboarding y las 7 vistas: Inicio, Practicar, Medallas, Grupos, Comunidad, Mi plan y Ajustes.
- **Backend:** Node.js + Express. Mismo contrato que la app y la consola admin (envelope `{data,error,meta}`, rutas `/api/v1`, JWT con claims `sub/role/plan`).
- **Banco de preguntas:** carga las **preguntas reales de la PAES** desde las carpetas `PAES Chile *` del repositorio (Comp. Lectora, Matematica M1, Ciencias). ~1.300 preguntas.

Estructura

```
backend/   API REST Express (rutas /api/v1/*) + driver de datos en memoria/JSON (Firestore opcional)
frontend/  SPA React (auth + onboarding + 7 vistas)
```

Driver de datos

El backend usa por defecto un **almacen en memoria persistido en `backend/.data/db.json`**, asi que corre sin Firebase. Para usar Cloud Firestore (mismo modelo que el doc) define en `.env` `DATA_DRIVER=firestore` e instala el SDK: `npm install firebase-admin` (con credenciales o emulador).

Puesta en marcha

1. Backend

```
cd backend
npm install
cp .env.example .env
npm run seed     # carga preguntas PAES reales + usuarios/grupos/planes/beneficios demo
npm start        # API en http://localhost:4100/api/v1
```

2. Frontend

```
cd frontend
npm install
npm run dev      # http://localhost:5174  (proxy /api -> :4100)
```

Usuarios de prueba (sembrados · contrasena `demo1234`)

Correo	Plan	Notas
demo@aprueba.cl	free	Cuota 5/10, racha 4, medallas variadas (muestra paywall y desbloqueo de cuota)
camila@correo.cl	all	Suscripcion anual activa, preguntas ilimitadas
juan@correo.cl	uni	Suscripcion mensual activa

Endpoints del alumno implementados

Auth: `POST /auth/register|login|social|refresh|logout`, `POST /auth/password/forgot|reset`. Perfil: `GET|PATCH|DELETE /me`, `GET|PATCH /me/settings`, `GET|PUT /me/preferences`, `GET /me/data-export`. Catalogo/onboarding: `GET /tests`, `GET /plans` (publico). Practica: `GET /practice/next`, `GET /questions/:id`, `POST /questions/:id/answer`, `GET /questions/:id/explanation`, `GET /questions/:id/skill`, `GET /me/quota`, `POST /me/quota/unlock`, `GET /me/progress`. Recorreccion: `POST|GET /corrections`. Medallas: `GET /me/medals`, `POST /me/medals/exchange`, `GET|POST /me/gifts`, `GET /benefits`, `POST /benefits/:id/redeem`. Grupos: `GET|POST /groups`, `GET|PATCH|DELETE /groups/:id`, invitaciones, miembros, `GET /groups/:id/stats|shared`, `POST /groups/:id/shared`. Comunidad: `GET /feed`, `POST /posts`, `POST /posts/:id/like`, `GET|POST /posts/:id/comments`. Planes/pagos: `POST /checkout/sessions` (+ `/:id/confirm` para la demo), `GET /me/subscription`, `POST /me/subscription/change|cancel`, `GET /me/invoices`, `POST /webhooks/stripe`. Notificaciones/dispositivos: `GET /me/notifications`, `PATCH /me/notifications/:id/read`, `PUT /me/notifications/preferences`, `POST /devices`, `DELETE /devices/:id`.

Todos respetan el envelope, los JWT con rol `student` y los codigos de error del documento de API.